

Logic and Sets 5

ordered binary decision diagrams



Today

binary decision trees

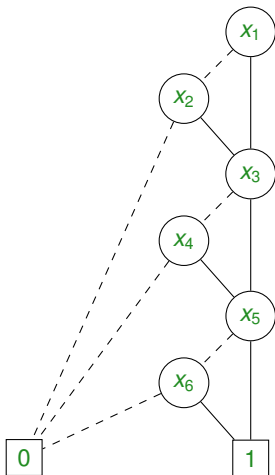
ordered binary decision diagrams (OBDDs)

reduced OBDDs

logical operations on OBDDs

universal and existential quantification

Ordered binary decision diagrams



Boolean functions

truth table

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Boolean function

$f(x, y)$
$f(1, 1) = 1$
$f(1, 0) = 0$
$f(0, 1) = 1$
$f(0, 0) = 1$

Each truth table expresses a *Boolean function*

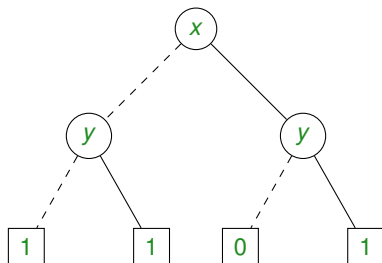
- ▶ from tuples (e.g. pairs, triples) of the truth values 1 and 0
- ▶ to the truth values 1 and 0.

Propositional variables p, q are now called *Boolean* variables x, y .

Binary decision tree

A Boolean function can be represented as a *binary decision tree*, with regard to some (total) order on its variables.

For example, for $x \rightarrow y$,
with x placed above y :



A *dashed* line (to the left) corresponds to the value 0, and a *solid* line (to the right) to the value 1, for the variable above.

Non-leaves are *circles*; they contain a *Boolean variable*.

Leaves are *boxes*; they contain 0 or 1.

Binary decision tree

Questions

Give the binary decision tree of the following formulas:

▶ x

▶ $\neg x$

▶ $x \rightarrow y$ (against the order y, x)

▶ $\top$

▶ $(\neg x \wedge z) \vee (x \wedge (y \oplus z))$ (against the order x, y, z)

Binary decision tree

Question

Suppose a propositional formula is *satisfiable*.

What does this mean for its binary decision tree ?

Question

Consider a propositional formula with n variables.

How many *leaves* does its binary decision tree contain ?

How many *rows* does its truth table contain ?

Ordered binary decision diagram

An *ordered binary decision diagram (OBDD)* is obtained by **collapsing** and **eliminating** nodes in a binary decision tree.

C1 All leaves labeled **0** are **collapsed**, and also all leaves labeled **1**.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the **0**- and **1**-edge of non-leaf n point to the *same* node n' ,

- * then n is **eliminated**, and
- * its incoming edges are **redirected** to n' .

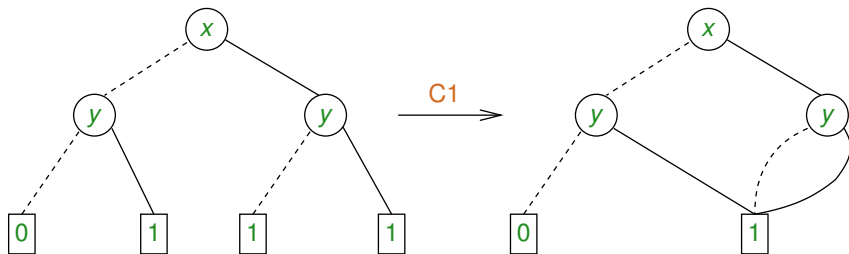
C3 (Two or more) non-leaves are **collapsed** to one node if:

- they are associated with the same variable, and
- their **0**-edges point to the same node, and their **1**-edges point to the same node.

Reducing OBDDs - Rule C1

Rule **C1**: **Collapse identical leaves.**

Example: Consider the binary decision tree for $x \vee y$.



Question

Collapse leaves in the binary decision tree for $x \rightarrow y$,
with y placed above x .

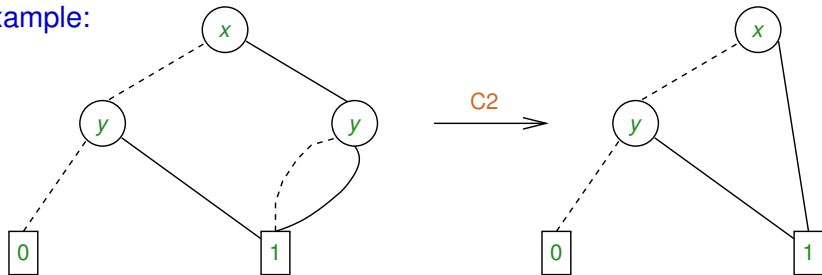
Reducing OBDDs - Rule C2

Rule **C2**: **Eliminate superfluous non-leaves.**

If both outgoing edges of a node n point to the same node n' , then:

- ▶ eliminate n , and
- ▶ redirect its incoming edges to n' .

Example:



Question

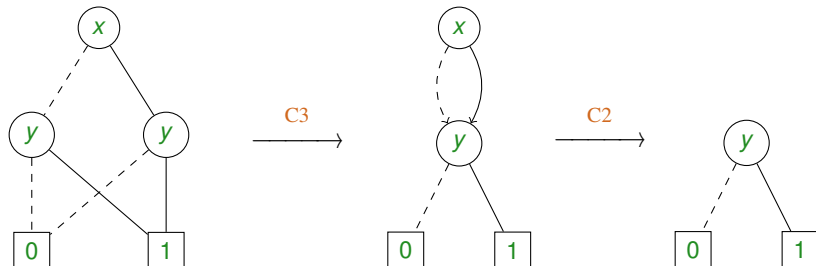
Eliminate superfluous non-leaves (after having applied **C1**) in the binary decision tree for $x \rightarrow y$, with y placed above x .

Reducing OBDDs - Rule C3

Rule **C3**: **Remove duplicate non-leaves.**

Collapse non-leaves associated with the same variable if their 0-edges and their 1-edges point to the same nodes.

Example: Consider the binary decision tree for $(x \wedge y) \vee (\neg x \wedge y)$, after collapsing its leaves with **C1**.



Reducing OBDDs

Theorem

The rules **C1**, **C2** and **C3** preserve semantic equivalence.

Question

Reduce the binary decision tree of

$$(\neg x \wedge z) \vee (x \wedge (y \oplus z))$$

using **C1**, **C2** and **C3**, until it cannot be reduced any further.

Take x, y, z as variable order.

Reduced OBDDs

Definition

An OBDD is called *reduced* if none of the rules C1, C2 and C3 can be applied to it.

Theorem

Each order in which the rules C2 and C3 are applied leads to the same reduced OBDD.

Question

What is the reduced OBDD for

- ▶ x
- ▶ a tautology

Checking semantic equivalence via reduced OBDDs

A binary decision tree of a formula corresponds to its truth table:
Each path from the root to a leaf captures one line in the truth table.

A reduced OBDD is a compressed representation of a truth table.

Theorem

Consider some order on variables.

Different (i.e., non-isomorphic) *reduced* OBDDs are never semantically equivalent.

So semantic equivalence of Boolean functions can be checked by comparing their *reduced* OBDDs, **against the same variable order**.

Applications of reduced OBDDs

Reduced OBDDs have widespread applications, for example:

- ▶ efficient synthesis of hardware circuits (logic synthesis)
- ▶ in formal methods, to symbolically represent state spaces of specified systems

Reduced OBDD for the even parity function

A reduced OBDD may provide a compact representation of a truth table, avoiding its exponential blow-up.

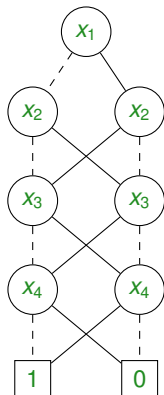
Example: *Parity functions* $f : \{0, 1\}^n \rightarrow \{0, 1\}$ are defined by:

$$f(x_1, \dots, x_n) = \begin{cases} 1 & \text{if an even number of } x_1, \dots, x_n \text{ are } 1 \\ 0 & \text{otherwise} \end{cases}$$

Consider the reduced OBDD if $n = 4$.

The variable order is x_1, x_2, x_3, x_4 .

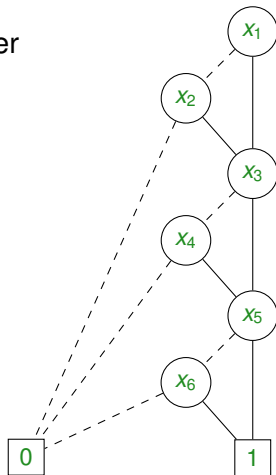
In general, the reduced OBDD has $2n + 1$ nodes, versus 2^n lines in the truth table.



OBDD representations are sensitive to order change

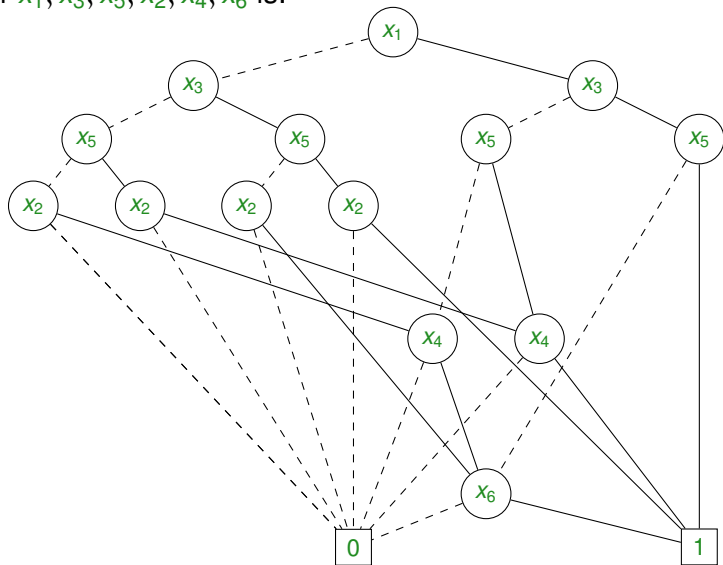
Example: Consider the CNF $(x_1 \vee x_2) \wedge (x_3 \vee x_4) \wedge (x_5 \vee x_6)$.

Its reduced OBDD for the variable order
 $x_1, x_2, x_3, x_4, x_5, x_6$ is:



OBDD representations are sensitive to order change

The reduced OBDD of $(x_1 \vee x_2) \wedge (x_3 \vee x_4) \wedge (x_5 \vee x_6)$ against the order $x_1, x_3, x_5, x_2, x_4, x_6$ is:



OBDD representations are sensitive to order change

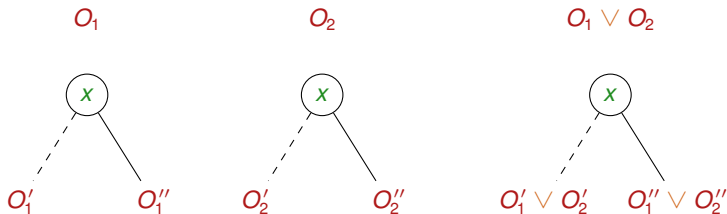
Given a Boolean function, determining a variable order that produces a reduced OBDD of minimal size is *NP-hard*.

Obtaining small reduced OBDDs is important for SAT solvers.

Because a reduced OBDD of a Boolean function (or a propositional formula) captures its truth table.

Challenge: Avoid the construction of a binary decision tree when building a reduced OBDD.

Logical operations on OBDDs



Logical operations on OBDDs - Negation

Question

Given an OBDD, how can we obtain an OBDD for the negation of the corresponding Boolean function ?

Answer: Swap 0 and 1 in the two leaf nodes.

If the original OBDD is reduced, then so is the resulting one.

Logical operations on OBDDs - Disjunction

Given OBDDs O_1 and O_2 , *with regard to the same variable order*.

We build an OBDD representing the *disjunction* of O_1 and O_2 .

Question

What to do if O_1 consists of a single leaf node ?

Answer: If O_1 is the leaf 0, then return O_2 .

If O_1 is the leaf 1, then return the leaf 1.

Likewise if O_2 consists of a single leaf node.

Logical operations on OBDDs - Disjunction

Let O_1 and O_2 *not* consist of a single leaf node.

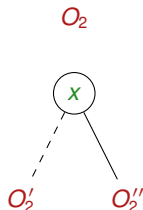
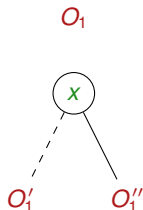
Let the 0-edge from the root of O_1 lead to the OBDD O'_1 .

Let the 1-edge from the root of O_1 lead to the OBDD O''_1 .

Let the 0-edge from the root of O_2 lead to the OBDD O'_2 .

Let the 1-edge from the root of O_2 lead to the OBDD O''_2 .

Case 1: The roots of O_1 and O_2 are associated to the same variable x .



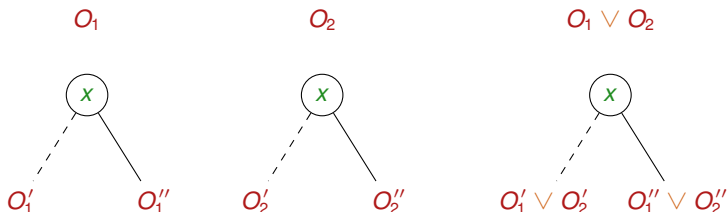
Logical operations on OBDDs - Disjunction

The root of the OBDD for $O_1 \vee O_2$ is associated to x .

So O'_1 and O'_2 are obtained from O_1 and O_2 by taking $x = 0$.

The 0-edge from the root of the OBDD for $O_1 \vee O_2$ leads to the OBDD for $O'_1 \vee O'_2$, which is computed recursively.

Likewise, the 1-edge from the root of the OBDD for $O_1 \vee O_2$ leads to the OBDD for $O''_1 \vee O''_2$, which is computed recursively.



Logical operations on OBDDs - Disjunction

Case 2: The roots of the non-leaves O_1 and O_2 are associated to *different* variables x and y , respectively.

Suppose x comes before y in the variable order.

Since x does not occur on O_2 , this OBDD is oblivious to x 's value.

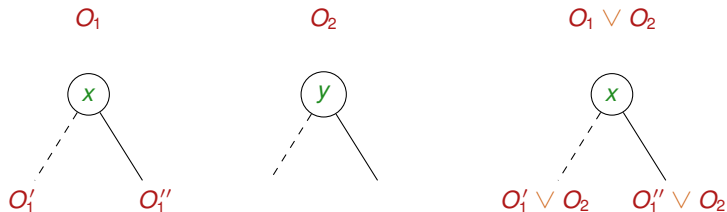
The root of the OBDD for $O_1 \vee O_2$ is associated to x .

The 0-edge from the root of the OBDD for $O_1 \vee O_2$ leads to the OBDD for $O_1' \vee O_2$, which is computed recursively.

Likewise, the 1-edge from the root of the OBDD for $O_1 \vee O_2$ leads to the OBDD for $O_1'' \vee O_2$, which is computed recursively.

Logical operations on OBDDs - Disjunction

x comes before y in the variable order.



Question

What to do if y comes before x in the variable order?

Logical operations on OBDDs - Disjunction

Given *reduced* OBDDs O_1 and O_2 , the constructed OBDD for $O_1 \vee O_2$ is *not* always reduced.

Question

Construct the following OBDDs.

In each case, reduce the resulting OBDD if needed.

- ▶ An OBDD for $x \vee \neg x$ from the OBDDs for x and $\neg x$.
- ▶ An OBDD for $x \vee y$ from the OBDDs for x and y , against the order x, y .

Logical operations on OBDDs - Conjunction

Question

Given OBDDs O_1 and O_2 , against the same variable order.
Build an OBDD representing the *conjunction* of O_1 and O_2 .

Answer: An OBDD for $O_1 \wedge O_2$ can be built in the same way as for $O_1 \vee O_2$ (with $\vee$ replaced by $\wedge$).

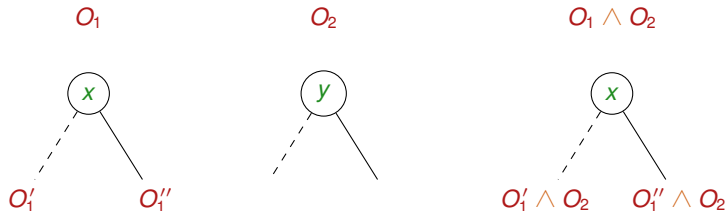
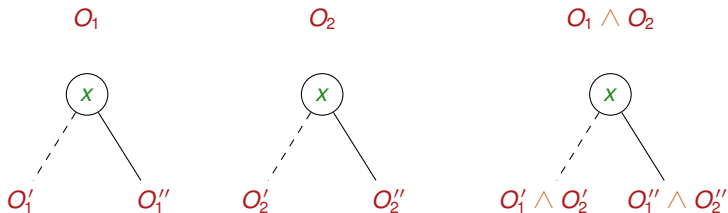
The only difference is if O_1 or O_2 consists of a single leaf node.

If O_1 is the leaf 0, then return the leaf 0.

If O_1 is the leaf 1, then return O_2 .

Likewise if O_2 consists of a single leaf node.

Logical operations on OBDDs - Conjunction



x comes before y in the variable order.

Logical operations on OBDDs - Conjunction

Question

Construct the following OBDDs.

In each case reduce the resulting OBDD if needed.

- ▶ An OBDD for $x \wedge \neg x$ from the OBDDs for x and $\neg x$.
- ▶ An OBDD for $(x \vee y) \wedge x$ from the reduced OBDDs for $x \vee y$ and x , against the order x, y .

Universal and existential quantification

Let x be a Boolean variable and ϕ a logic formula.

$\forall x \phi$ is 1 (i.e., true) if:

- ▶ ϕ with 1 substituted for x yields 1, **and**
- ▶ ϕ with 0 substituted for x yields 1.

In other words, ϕ is true for *all* values of x .

$\exists x \phi$ is 1 if:

- ▶ ϕ with 1 substituted for x yields 1, **or**
- ▶ ϕ with 0 substituted for x yields 1.

In other words, ϕ is true for *some* value of x .

Universal and existential quantification

Examples

- ▶ $\exists x \ x$ is a tautology (take x to be 1)
- ▶ $\forall x \ x$ is a contradiction (take x to be 0)
- ▶ $\forall x \ (x \rightarrow y) \equiv y$
- ▶ $\exists x \ \forall y \ (x \wedge y)$ is a contradiction (take y to be 0)
- ▶ $\exists x \ \forall y \ (x \vee y)$ is a tautology (take x to be 1)

A truth table

If $y = 1$, then $x \rightarrow y$ holds for both $x = 1$ and $x = 0$,
so $\forall x (x \rightarrow y)$ holds.

If $y = 0$, then $x \rightarrow y$ does *not* hold for $x = 1$,
so $\forall x (x \rightarrow y)$ does *not* hold.

y	$\forall x (x \rightarrow y)$
1	1
0	0

Hence $\forall x (x \rightarrow y) \equiv y$.

There is no column for x because its occurrence in $\forall x (x \rightarrow y)$ is *bound* by the universal quantification.

Universal and existential quantification

Question

What is the meaning of the following formulas ?

► $\forall x \exists y (x \oplus y)$

► $\exists y \forall x (x \oplus y)$

Answer: The first formula is a tautology.

The second formula is a contradiction.

$\forall x \exists y \phi$ allows a *different* y for each different x .

$\exists y \forall x \phi$ requires the *same* y for each different x .

Expressing quantification in propositional logic

Consider a propositional formula ϕ .

Let ϕ_1 and ϕ_0 denote ϕ with 1 and 0 substituted for x , respectively.

$$\forall x \phi \equiv \phi_1 \wedge \phi_0$$

Question

How can $\exists x \phi$ be expressed in propositional logic ?

Answer:

$$\exists x \phi \equiv \phi_1 \vee \phi_0$$

Universal quantification on OBDDs

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect incoming edges of an eliminated node to the 1-child of this node. Or, if x is at the root, make its 1-child the root.

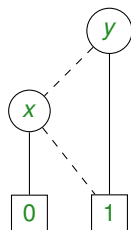
We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect incoming edges of an eliminated node to the 0-child of this node. Or, if x is at the root, make its 0-child the root.

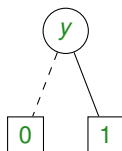
$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

Universal quantification on OBDDs

Example: We build an OBDD for $\forall x (x \rightarrow y)$, against the order y, x .



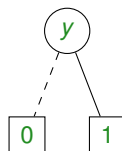
$x \rightarrow y$



$x = 1$



$x = 0$



$\forall x (x \rightarrow y)$

So $\forall x (x \rightarrow y) \equiv y$.

Existential quantification on OBDDs

Question

Given an OBDD O , build an OBDD representing $\exists x O$.

Answer: An OBDD for $\exists x O$ can be built in the same way as for $\forall x O$.

The only difference is that it is captured by the OBDD for $O(1) \vee O(0)$.

Question

Build reduced OBDDs for the following formulas, against the order x, y :

► $\exists x (x \rightarrow y)$

► $\forall x (x \wedge y)$

► $\forall x (x \oplus y)$

► $\exists x (x \oplus y)$

Take home

binary decision trees

OBDDs

reduced OBDDs (compressed representations of truth tables)

logical operations on OBDDs

universal and existential quantification